WrappedNLP

Inherits: ERC20, Ownable2Step, ReentrancyGuardTransient

Redemption Options:

- Queue-based: Free redemption via WithdrawQueue contract (time-delayed for JIT protection)
- Instant: Direct redemption with configurable fee (immediate, open to all users)

Converts rebasing NativeLPToken into static-balance ERC20 for protocol compatibility while preserving accumulated yield value.

State Variables

instantRedeemEnabled

Whether instant redeem is enabled

```
bool public instantRedeemEnabled = true;
```

nlp

The Native LPToken address

```
NativeLPToken public immutable nlp;
```

underlying

The underlying token of the NativeLPToken

```
IERC20 public immutable underlying;
```

feeRecipient

Address that receives instant redeem fees

```
address public feeRecipient;
```

withdrawQueue

The authorized WithdrawQueue contract

```
WithdrawQueue public withdrawQueue;
```

MIN_INSTANT_REDEEM_AMOUNT

Minimum amount for instant redeem

```
uint256 public constant MIN_INSTANT_REDEEM_AMOUNT = 1000;
```

instantRedeemFeeBips

Instant redeem fee in basis points (1 bip = 0.01%)

```
uint256 public instantRedeemFeeBips;
```

redeemWhitelist

Mapping of whitelisted addresses that can directly call unwrapAndRedeem

```
mapping(address => bool) public redeemWhitelist;
```

Functions

constructor

```
constructor(string memory _name, string memory _symbol, NativeLPToken _nlp) ERC20(_name,
_symbol);
```

wrap

Wraps NativeLP tokens into WrappedNLP tokens

```
function wrap(uint256 amount) external nonReentrant returns (uint256 wNLPAmount);
```

Parameters

Name	Type	Description
`amount`	`uint256`	Amount of NativeLP tokens to deposit

Returns

Name	Type	Description
`wNLPAmount`	`uint256`	Amount of WrappedNLP tokens received

depositAndWrap

Deposit underlying tokens directly into NLP and receive WrappedNLP tokens for a specified recipient

```
function depositAndWrap(address to, uint256 amount) external nonReentrant returns (uint256 wNLPAmount);
```

Parameters

Name	Type	Description
`to`	`address`	The address to receive the WrappedNLP tokens
`amount`	`uint256`	Amount of underlying tokens to deposit

Returns

Name	Type	Description
`wNLPAmount`	`uint256`	Amount of WrappedNLP tokens received

unwrapAndRedeem

Unwrap WrappedNLP tokens and redeem underlying tokens to a specified address

Only WithdrawQueue contract or whitelisted addresses can call this function

```
function unwrapAndRedeem(uint256 wNLPAmount, address to) external nonReentrant returns (uint256 underlyingAmount);
```

Parameters

Name	Type	Description
`wNLPAmount`	`uint256`	Amount of WrappedNLP tokens to unwrap and redeem
`to`	`address`	Address that will receive the underlying tokens

Returns

Name	Type	Description
`underlyingAmount`	`uint256`	Amount of underlying tokens sent to the recipient

instantRedeem

Instantly redeem WrappedNLP tokens for underlying tokens with fee

```
function instantRedeem(uint256 wNLPAmount, address to) external nonReentrant returns (uint256 underlyingAmount);
```

Parameters

Name	Type	Description
`wNLPAmount`	`uint256`	Amount of WrappedNLP tokens to redeem
`to`	`address`	Address that will receive the underlying tokens

Returns

Name	Type	Description
`underlyingAmount`	`uint256`	Amount of underlying tokens received after fee

decimals

Gets the number of decimals for this token

```
function decimals() public view override returns (uint8);
```

Returns

Name	Type	Description
`<none>`	`uint8`	The number of decimals, matching the underlying token

getWnlpByNlp

Calculates the amount of WrappedNLP tokens for a given amount of NativeLP tokens

```
function getWnlpByNlp(uint256 _nlpAmount) external view returns (uint256);
```

Parameters

Name	Type	Description
`_nlpAmount`	`uint256`	Amount of NativeLP tokens

Returns

Name	Type	Description
`<none>`	`uint256`	Amount of WrappedNLP tokens

getNlpByWnlp

Calculates the amount of NLP tokens for a given amount of WrappedNLP tokens

```
function getNlpByWnlp(uint256 wNLPAmount) external view returns (uint256);
```

Parameters

Name	Type	Description
`wNLPAmount`	`uint256`	Amount of WrappedNLP tokens

Returns

Name	Type	Description
`<none>`	`uint256`	Amount of NativeLP tokens

nlpPerToken

Returns the amount of NativeLP tokens per 1 WrappedNLP token

Used to calculate the exchange rate from WrappedNLP to NativeLP

```
function nlpPerToken() external view returns (uint256);
```

Returns

Name	Type	Description
`<none>`	`uint256`	Amount of NativeLP tokens equivalent to 1 WrappedNLP token

tokensPerNlp

Returns the amount of WrappedNLP tokens per 1 NativeLP token

Used to calculate the exchange rate from NativeLP to WrappedNLP

```
function tokensPerNlp() external view returns (uint256);
```

Returns

Name	Type	Description
`<none>`	`uint256`	Amount of WrappedNLP tokens equivalent to 1 NativeLP token

setWithdrawQueue

Sets the WithdrawQueue contract address

```
function setWithdrawQueue(address _withdrawQueue) external onlyOwner;
```

Parameters

Name	Type	Description
`_withdrawQueue`	`address`	The new WithdrawQueue contract address

setRedeemWhitelist

Sets the redeem whitelist status for multiple addresses

```
function setRedeemWhitelist(address[] calldata accounts, bool[] calldata statuses)
external onlyOwner;
```

Parameters

Name	Type	Description
`accounts`	`address[]`	The addresses to update
`statuses`	`bool[]`	The new whitelist statuses corresponding to each account

setFeeRecipient

Sets the address that receives instant redeem fees

```
function setFeeRecipient(address _feeRecipient) external onlyOwner;
```

Parameters

Name	Type	Description
`_feeRecipient`	`address`	The new fee recipient address

setInstantRedeemFeeBips

Sets the instant redeem fee in basis points

```
function setInstantRedeemFeeBips(uint256 _instantRedeemFeeBips) external onlyOwner;
```

Parameters

Name	Type	Description
`_instantRedeemFeeBips`	`uint256`	The new instant redeem fee in basis points

setInstantRedeemEnabled

Sets whether instant redeem is enabled

```
function setInstantRedeemEnabled(bool _enabled) external onlyOwner;
```

Parameters

Name	Type	Description
`_enabled`	`bool`	Whether instant redeem should be enabled

Events

Wrapped

Event emitted when NativeLP tokens are wrapped into WrappedNLP tokens

```
event Wrapped(address indexed user, uint256 nlpAmount, uint256 wNLPAmount);
```

DepositAndWrapped

Event emitted when underlying tokens are deposited and wrapped into WrappedNLP tokens

```
event DepositAndWrapped(address indexed from, address indexed to, uint256
underlyingAmount, uint256 wNLPAmount);
```

UnwrappedAndRedeemed

Event emitted when WrappedNLP tokens are unwrapped and redeemed for underlying tokens

```
event UnwrappedAndRedeemed(address indexed from, address indexed to, uint256 wNLPAmount,
uint256 underlyingAmount);
```

InstantRedeemFeeCollected

Event emitted when instant redeem fee is charged

```
event InstantRedeemFeeCollected(address indexed user, uint256 feeAmount);
```

FeeRecipientUpdated

Event emitted when fee recipient is updated

```
event FeeRecipientUpdated(address indexed oldRecipient, address indexed newRecipient);
```

InstantRedeemed

Event emitted when instant redeem is executed

```
event InstantRedeemed(address indexed user, uint256 wNLPAmount, uint256 underlyingAmount,
uint256 feeAmount);
```

WithdrawQueueUpdated

Event emitted when withdraw queue address is updated

```
event WithdrawQueueUpdated(address indexed oldQueue, address indexed newQueue);
```

RedeemWhitelistUpdated

Event emitted when redeem whitelist status is updated

```
event RedeemWhitelistUpdated(address indexed account, bool status);
```

InstantRedeemFeeBipsUpdated

Event emitted when instant redeem fee bips is updated

```
event InstantRedeemFeeBipsUpdated(uint256 oldFeeBips, uint256 newFeeBips);
```

InstantRedeemEnabledUpdated

Event emitted when instant redeem enabled status is updated

```
event InstantRedeemEnabledUpdated(bool enabled);
```